

실험 3호



Llama Fine-Tuning

Team 내 라마짱이 이렇게 똑똑할 리가 없어
정유진 임민석



▼ 목차

1. 환경 설정

2. LoRA 실험

3. 결과 및 분석

4. 결론

▼ 1. 환경 설정

서버 연결

ssh adminuser@203.246.112.74

mjstrong1212@@

10427

- ssh -p 10427 adminuser@203.246.112.74

날아가서 슬픈...

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f0bfc009ad41	device_2	"/opt/nvidia/nvidia_..."	13 hours ago	Exited (129) 5 hours ago		snagwon_3
396cb920d92f	backup:image	"/opt/nvidia/nvidia_..."	13 hours ago	Exited (127) 13 hours ago		device_2
1ab502211e90	pytorch/pytorch:2.1.0-cuda11.8-cudnn8-runtime	"bash"	17 hours ago	Exited (129) 17 hours ago		clever_wilson
7e8c72327e61	rian	"/opt/nvidia/nvidia_..."	17 hours ago	Exited (0) 13 hours ago		rain
4b9dbb98b1fd	pytorch/pytorch:2.3.1-cuda11.8-cudnn8-runtime	"bash"	18 hours ago	Up 10 hours		TeamB-C
12dda4798dae	lora-train:cull6	"/opt/nvidia/nvidia_..."	34 hours ago	Up 19 hours		device_3
ba908076b9e5	device_2	"/opt/nvidia/nvidia_..."	35 hours ago	Up 13 hours		sw_jh_2
d572b693dcbb	lora_llama_cuda116	"/opt/nvidia/nvidia_..."	35 hours ago	Exited (129) 31 hours ago		sad_kare

Docker & Conda

도커 다시 만들기

- `docker run -it --name strong_minseok --gpus '"device=0"' pytorch/pytorch:2.3.1-cuda11.8-cudnn8-runtime bash`

```
adminuser@ubuntu:~$ docker run -it --name strong_minseok --gpus '"device=0"' pytorch/pytorch:2.3.1-cuda11.8-cudnn8-runtime bash
root@49bc6c4e4fa4:/workspace# exit
```

도커 들어가기

- `docker exec -it strong_minseok bash`
- `docker start -ai strong_minseok`

```
adminuser@ubuntu:~$ docker start -ai strong_minseok
root@49bc6c4e4fa4:/workspace#
```

버전 확인

- `conda --version`

```
root@49bc6c4e4fa4:/workspace# conda --version
conda 23.5.2
```

콘다 환경 만들기

- `conda create -n experiment1 python=3.10 -y`

썬다 키기

```
root@49bc6c4e4fa4:/workspace# conda activate experiment1
CommandNotFoundError: Your shell has not been properly configured to use 'conda activate'.
To initialize your shell, run

    $ conda init <SHELL_NAME>

Currently supported shells are:
- bash
- fish
- tcsh
- xonsh
- zsh
- powershell

See 'conda init --help' for more information and options.
IMPORTANT: You may need to close and restart your shell after running 'conda init'.
```

```
root@49bc6c4e4fa4:/workspace# conda env list
# conda environments:
#
base                /opt/conda
experiment1         /opt/conda/envs/experiment1
```

- source /opt/conda/etc/profile.d/conda.sh
- conda activate experiment1

```
root@49bc6c4e4fa4:/workspace# source /opt/conda/etc/profile.d/conda.sh
conda activate experiment1
(experiment1) root@49bc6c4e4fa4:/workspace# |
```

라이브러리 설치

파이썬

- conda install -y python=3.10

```
(experiment1) root@49bc6c4e4fa4:/workspace# python --version
Python 3.10.19
(experiment1) root@49bc6c4e4fa4:/workspace# conda install -y python=3.10
Collecting package metadata (current_repodata.json): done
Solving environment: done

==> WARNING: A newer version of conda exists. <==
  current version: 23.5.2
  latest version: 25.11.1

Please update conda by running

  $ conda update -n base -c defaults conda

Or to minimize the number of packages updated during conda update use

  conda install conda=25.11.1

# All requested packages already installed.
```

pip

- python -m pip install -U pip setuptools wheel

```
(experiment1) root@49bc6c4e4fa4:/workspace# python -m pip install -U pip setuptools wheel
Requirement already satisfied: pip in /opt/conda/envs/experiment1/lib/python3.10/site-packages (25.3)
Requirement already satisfied: setuptools in /opt/conda/envs/experiment1/lib/python3.10/site-packages (80.9.0)
Requirement already satisfied: wheel in /opt/conda/envs/experiment1/lib/python3.10/site-packages (0.45.1)
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
```

torch 2.3.1

- pip install torch==2.3.1 --index-url <https://download.pytorch.org/whl/cu118>

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install torch==2.3.1 --index-url https://download.pytorch.org/whl/cu118
Looking in indexes: https://download.pytorch.org/whl/cu118
Collecting torch==2.3.1
  Downloading https://download.pytorch.org/whl/cu118/torch-2.3.1%2Bcu118-cp310-cp310-linux_x86_64.whl (839.7 MB)
    839.7/839.7 MB 41.1 MB/s 0:00:13
Collecting filelock (from torch==2.3.1)
  Downloading filelock-3.20.0-py3-none-any.whl.metadata (2.1 kB)
Collecting typing-extensions>=4.8.0 (from torch==2.3.1)
  Downloading https://download.pytorch.org/whl/typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting sympy (from torch==2.3.1)
  Downloading sympy-1.14.0-py3-none-any.whl.metadata (12 kB)
Collecting networkx (from torch==2.3.1)
  Downloading networkx-3.4.2-py3-none-any.whl.metadata (6.3 kB)
Collecting Jinja2 (from torch==2.3.1)
  Downloading https://d21usjoq99fcb9.cloudfront.net/whl/jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting fsspec (from torch==2.3.1)
  Downloading fsspec-2025.12.0-py3-none-any.whl.metadata (10 kB)
Collecting nvidia-cuda-nvrtc-cu11==11.8.89 (from torch==2.3.1)
  Downloading https://d21usjoq99fcb9.cloudfront.net/whl/cu118/nvidia_cuda_nvrtc_cu11-11.8.89-py3-none-manylinux1_x86_64.whl (23.2 MB)
    23.2/23.2 MB 77.6 MB/s 0:00:00
Collecting nvidia-cuda-runtime-cu11==11.8.89 (from torch==2.3.1)
  Downloading https://d21usjoq99fcb9.cloudfront.net/whl/cu118/nvidia_cuda_runtime_cu11-11.8.89-py3-none-manylinux1_x86_64.whl (23.2 MB)
    23.2/23.2 MB 77.6 MB/s 0:00:00
```

- python -c "import torch; print(torch.**version**, torch.cuda.is_available(), torch.version.cuda)"

```
(experiment1) root@49bc6c4e4fa4:/workspace# python -c "import torch; print(torch.__version__, torch.cuda.is_available(), torch.version.cuda)"
2.3.1+cu118 True 11.8
```

transformers 4.44.2

- pip install transformers==4.44.2

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install transformers==4.44.2
Collecting transformers==4.44.2
  Downloading transformers-4.44.2-py3-none-any.whl.metadata (43 kB)
Requirement already satisfied: filelock in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from transformers==4.44.2) (3.20.0)
Collecting huggingface-hub<1.0, >=0.23.2 (from transformers==4.44.2)
  Downloading huggingface_hub-0.36.0-py3-none-any.whl.metadata (14 kB)
Collecting numpy>=1.17 (from transformers==4.44.2)
```

- pip install tokenizers sentencepiece

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install tokenizers sentencepiece
Requirement already satisfied: tokenizers in /opt/conda/envs/experiment1/lib/python3.10/site-packages (0.19.1)
Collecting sentencepiece
  Downloading sentencepiece-0.2.1-cp310-cp310-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (10 kB)
Requirement already satisfied: huggingface-hub<1.0,>=0.16.4 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from tokenizers) (0.36.0)
Requirement already satisfied: filelock in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.16.4->tokenizers) (3.20.0)
Requirement already satisfied: fsspec>=2023.5.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.16.4->tokenizers) (2025.12.0)
Requirement already satisfied: packaging>=20.9 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.16.4->tokenizers) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.16.4->tokenizers) (6.0.3)
Requirement already satisfied: requests in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub<1.0,>=0.16.4->tokenizers) (2.32.5)
```

accelerate 0.33.0

- pip install accelerate==0.33.0

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install accelerate==0.33.0
Collecting accelerate==0.33.0
  Downloading accelerate-0.33.0-py3-none-any.whl.metadata (18 kB)
Collecting numpy<2.0.0,>=1.17 (from accelerate==0.33.0)
  Downloading numpy-1.26.4-cp310-cp310-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (61 kB)
Requirement already satisfied: packaging>=20.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from accelerate==0.33.0) (25.0)
Collecting psutil (from accelerate==0.33.0)
  Downloading psutil-7.2.1-cp36-abi3-manylinux2010_x86_64.manylinux_2_12_x86_64.manylinux_2_28_x86_64.whl.metadata (22 kB)
Requirement already satisfied: pyyaml in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from accelerate==0.33.0) (6.0.3)
Requirement already satisfied: torch>=1.10.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from accelerate==0.33.0) (2.3.1+cu118)
Requirement already satisfied: huggingface-hub>=0.21.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from accelerate==0.33.0) (0.36.0)
Requirement already satisfied: safetensors>=0.3.1 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from accelerate==0.33.0) (0.7.0)
Requirement already satisfied: filelock in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub>=0.21.0->accelerate==0.33.0) (3.20.0)
Requirement already satisfied: fsspec>=2023.5.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub>=0.21.0->accelerate==0.33.0) (2025.12.0)
```

peft 0.12.0

- pip install peft==0.12.0

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install peft==0.12.0
Collecting peft==0.12.0
  Downloading peft-0.12.0-py3-none-any.whl.metadata (13 kB)
Requirement already satisfied: numpy>=1.17 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from peft==0.12.0) (1.26.4)
Requirement already satisfied: packaging>=20.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from peft==0.12.0) (25.0)
Requirement already satisfied: psutil in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from peft==0.12.0) (7.2.1)
Requirement already satisfied: pyyaml in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from peft==0.12.0) (6.0.3)
Requirement already satisfied: torch>=1.13.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from peft==0.12.0) (2.3.1+cu118)
```

datasets 2.21.0

- pip install datasets==2.21.0

```
(experiment1) root@49bc6c4e4fa4:/workspace# pip install datasets==2.21.0
Collecting datasets==2.21.0
  Downloading datasets-2.21.0-py3-none-any.whl.metadata (21 kB)
Requirement already satisfied: filelock in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from datasets==2.21.0) (3.20.0)
Requirement already satisfied: numpy>=1.17 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from datasets==2.21.0) (1.26.4)
Collecting pyarrow>=15.0.0 (from datasets==2.21.0)
  Downloading pyarrow-23.0.0-cp310-cp310-manylinux_2_28_x86_64.whl.metadata (3.0 kB)
Collecting dill<0.3.9,>=0.3.0 (from datasets==2.21.0)
  Downloading dill-0.3.8-py3-none-any.whl.metadata (10 kB)
Collecting pandas (from datasets==2.21.0)
  Downloading pandas-2.3.3-cp310-cp310-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (91 kB)
Requirement already satisfied: requests>=2.32.2 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from datasets==2.21.0) (2.32.5)
Requirement already satisfied: tqdm>=4.66.3 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from datasets==2.21.0) (4.67.1)
Collecting xxhash (from datasets==2.21.0)
```

버전 체크

```
python - << 'PY'
import torch, transformers, accelerate, peft, datasets
print("torch", torch.__version__)
print("cuda_available", torch.cuda.is_available())
print("cuda_version", torch.version.cuda)
print("transformers", transformers.__version__)
print("accelerate", accelerate.__version__)
print("peft", peft.__version__)
print("datasets", datasets.__version__)
PY
```

```
(experiment1) root@49bc6c4e4fa4:/workspace# python - << 'PY'
import torch, transformers, accelerate, peft, datasets
print("torch", torch.__version__)
print("cuda_available", torch.cuda.is_available())
print("cuda_version", torch.version.cuda)
print("transformers", transformers.__version__)
print("accelerate", accelerate.__version__)
print("peft", peft.__version__)
print("datasets", datasets.__version__)
PY

torch 2.3.1+cu118
cuda_available True
cuda_version 11.8
transformers 4.44.2
accelerate 0.33.0
peft 0.12.0
datasets 2.21.0
```

Huggingface

새로 발급 받은 토큰

- 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
- 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 - 익명의 것

Huggingface CLI 설치

- `pip install -U huggingface_hub`

```
(Experiment1) root@49b6c64e4fa0:/workspace# pip install -U "huggingface-hub[cli]==0.24.7"
Requirement already satisfied: huggingface-hub==0.24.7 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub[cli]==0.24.7) (0.24.7)
Requirement already satisfied: filelock in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (3.20.0)
Requirement already satisfied: fsspec>=2023.5.0 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (2024.6.1)
Requirement already satisfied: packaging>=20.9 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (25.0)
Requirement already satisfied: pyyaml>=5.1 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (6.0.3)
Requirement already satisfied: requests in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (2.32.5)
Requirement already satisfied: tqdm>=4.42.1 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (4.67.1)
Requirement already satisfied: typing-extensions>=3.7.4.3 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub==0.24.7->huggingface-hub[cli]==0.24.7) (4.15.0)
Requirement already satisfied: InquirerPy==0.3.4 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from huggingface-hub[cli]==0.24.7) (0.3.4)
Requirement already satisfied: pty>=0.4.0, >=0.3.1 in /opt/conda/envs/experiment1/lib/python3.10/site-packages (from InquirerPy==0.3.4->huggingface-hub[cli]==0.24.7) (0.3.4.0)
```

hf version

```
(experiment1) root@49bc6c4e4fa4:/workspace# hf version
1.3.2
```

확인

```
(experiment1) root@49bc6c4e4fa4:/workspace# python -c "import huggingface_hub; print('hub', huggingface_hub.__version__)"
python -c "import transformers; print('transformers', transformers.__version__)"
hub 0.24.7
transformers 4.44.2
```

로그인

- huggingface-cli login

[illegible]

버전 확인

```
(experiment1) root@49bc6c4e4fa4:/workspace# python -c "import huggingface_hub; print(huggingface_hub.__version__)"
0.24.7
```

사용자 확인

```
(experiment1) root@49bc6c4e4fa4:/workspace# huggingface-cli whoami
AlicexWonder
```

▼ 2. LoRA 실험

2.1 실험 목적

LoRA Rank 설정에 따른 성능 분석

- LoRA 논문에서 보고된 low-rank($r=8$)의 효율성과 high-rank($r=64$)의 성능 저하 경향을 GSM8K 태스크에서 검증한다.

Query-Value vs Query-Key-Value 비교 실험

- LoRA 논문에서는 Query와 Value projection 조합이 가장 효과적인 설정으로 보고되었으나, Key projection을 포함했을 때의 영향은 명확히 분석되지 않았다.
- Attention Value 계산에 Query, Key, Value가 모두 사용됨에도 불구하고 Query-Value 조합이 우수한 성능을 보이는 이유를 분석하기 위해, Key projection을 추가한 경우와의 성능을 비교한다.

추가 실험) Attention vs MLP 비교 실험

- Attention(qv, qkv)과 MLP에 각각 LoRA를 적용하였을 때 어느 쪽의 성능이 우수한지 비교한다.

2.2 실험 설계

A. 하드웨어, 소프트웨어 환경

- GPU : V 100 32GB
- Framework : HuggingFace Transformer
- Precision : fp 16

B. 모델 설정

- Model : Llama-3.1-8B
- Base model vs LoRA fine-tuned
- LoRA rank : $r=8, 64$
- Target modules : W_qW_v , $W_qW_kW_v$, All, MLP

C. Training Hyperparameters

- Epoch = 3
- Learning rate :
 - $r = 8 \rightarrow 1e-4$
 - $r = 64 \rightarrow 2e-5$
 - 파라미터가 너무 많을때 학습률도 높으면 안정성이 떨어져서 조절

D. Dataset

- GSM8K
- Size : 5.89 MB

```
Weng earns $12 an hour for babysitting. Yesterday, she just did 50 minutes of babysitting. How much did she earn?
```

```
Weng earns  $12/60 = \$\langle 12/60=0.2 \rangle 0.2$  per minute.  
Working 50 minutes, she earned  $0.2 \times 50 = \$\langle 0.2*50=10 \rangle 10$ .  
#### 10
```

2.3 구현 사항

- 하이퍼파라미터 4개
 - $r=8, r=64, W_qW_v, W_qW_vW_k$
- 학습은 4개를 한꺼번에 → 추론은 5개를 한꺼번에 할 수 있도록 구현

- 시간 측정 기능
- 로깅 기능

2.4 실험 개요

A. 학습

- $r=8$, $WqWv$
- $r=8$, $WqWvWk$
- $r=64$, $WqWv$
- $r=64$, $WqWvWk$

B. 추론

- baseline 실험
- $r=8$, $WqWv$
- $r=8$, $WqWvWk$
- $r=64$, $WqWv$
- $r=64$, $WqWvWk$

2.5 초기 모델 코드

▼ 코드 1

```
import torch
import time
import logging
import json
import gc
from datasets import load_dataset
from transformers import (
    AutoModelForCausalLM,
    AutoTokenizer,
    TrainingArguments,
    Trainer,
```

```

        DataCollatorForSeq2Seq
    )
    from peft import LoraConfig, get_peft_model, TaskType
    e, PeftModel

# 1. 로깅 설정
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(levelname)s - %(message)s',
    handlers=[logging.FileHandler("experiment.log"),
              logging.StreamHandler()]
)
logger = logging.getLogger(__name__)

# 2. 환경 설정 및 하이퍼파라미터
MODEL_ID = "meta-llama/Llama-3.1-8B"
DATASET_NAME = "gsm8k"
DATASET_CONFIG = "main"
OUTPUT_DIR = "./results"

# 실험 매트릭스
configs = [
    {"r": 8, "target_modules": ["q_proj", "v_proj"],
     "lr": 1e-4, "name": "r8_qv"},
    {"r": 8, "target_modules": ["q_proj", "k_proj",
                               "v_proj"], "lr": 1e-4, "name": "r8_qkv"},
    {"r": 64, "target_modules": ["q_proj", "v_proj"],
     "lr": 2e-5, "name": "r64_qv"},
    {"r": 64, "target_modules": ["q_proj", "k_proj",
                               "v_proj"], "lr": 2e-5, "name": "r64_qkv"},
]

def preprocess_function(examples, tokenizer):
    """
    데이터 전처리 함수
    - 입력: Question
    - 정답: Answer + EOS Token

```

```

- Label: Pad 토큰은 -100으로 마스킹
"""
    inputs = [f"Question: {q}\nAnswer: " for q in examples["question"]]
    # 모델이 끝나는 지점을 알 수 있도록 EOS 토큰 추가
    targets = [f"{a} {tokenizer.eos_token}" for a in examples["answer"]]

    model_inputs = tokenizer(inputs, max_length=512, truncation=True, padding="max_length")
    labels = tokenizer(targets, max_length=512, truncation=True, padding="max_length")

    # 패딩 토큰의 라벨을 -100으로 설정 (Loss 계산 제외)
    labels["input_ids"] = [
        [(l if l != tokenizer.pad_token_id else -100) for l in label] for label in labels["input_ids"]
    ]
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

def run_experiment():
    # 토크나이저 로드
    tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
    tokenizer.pad_token = tokenizer.eos_token

    # -----
    # [중요] 데이터셋 분할 (Data Leakage 방지)
    # -----

    logger.info("Loading and splitting dataset...")
    raw_dataset = load_dataset(DATASET_NAME, DATASET_CONFIG)

    # Train 셋을 9:1로 나누어 실제 학습용(train)과 검증용(validation)으로 분리

```

```

split_dataset = raw_dataset["train"].train_test_s
plit(test_size=0.1, seed=42)
raw_train_dataset = split_dataset["train"]
raw_eval_dataset = split_dataset["test"] # 학습
중 Best Model 선정용
raw_test_dataset = raw_dataset["test"] # 최종
추론 및 성능 보고용 (학습 절대 미참여)

# 전처리 적용
train_dataset = raw_train_dataset.map(lambda x: p
reprocess_function(x, tokenizer), batched=True)
eval_dataset = raw_eval_dataset.map(lambda x: pre
process_function(x, tokenizer), batched=True)

results_summary = {}

# --- A. 4가지 LoRA 설정 학습 루프 ---
for cfg in configs:
    logger.info(f"--- Starting Training: {cfg['na
me']} ---")
    start_time = time.time()

    # 메모리 초기화
    torch.cuda.empty_cache()
    gc.collect()

    # 모델 로드 (FP16, Standard LoRA)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        torch_dtype=torch.float16,
        device_map="auto",
        use_cache=False # Gradient Checkpointing
사용 시 False 필수
    )

    # Gradient Checkpointing 활성화 (VRAM 절약 핵심)
    model.gradient_checkpointing_enable()
    model.enable_input_require_grads()

```

```

# LoRA Config 설정
lora_config = LoraConfig(
    r=cfg["r"],
    lora_alpha=cfg["r"] * 2,
    target_modules=cfg["target_modules"],
    lora_dropout=0.05,
    bias="none",
    task_type=TaskType.CAUSAL_LM
)
model = get_peft_model(model, lora_config)
model.print_trainable_parameters()

# 학습 인자 설정 (Best Model 저장 로직 포함)
training_args = TrainingArguments(
    output_dir=f"{OUTPUT_DIR}/{cfg['name']}",

    # 메모리 안전 설정
    per_device_train_batch_size=1,
    gradient_accumulation_steps=16, # 실질적 배
    ch 사이즈 16

    learning_rate=cfg["lr"],
    num_train_epochs=1, # 빠른 실험을 위해 1 epo
    ch (필요시 조정)
    fp16=True,

    # --- 평가 및 Best Model 저장 설정 ---
    evaluation_strategy="steps",
    eval_steps=50, # 50 스텝마다 검
    증

    save_strategy="steps",
    save_steps=50, # eval_steps와
    동일하게 설정

    load_best_model_at_end=True, # 학습 종료 시
    가장 Loss 낮은 모델 로드
    metric_for_best_model="loss",
    save_total_limit=1, # 용량 절약을 위

```

해 Best 1개만 유지

```
# 평가 시 OOM 방지
per_device_eval_batch_size=1,
eval_accumulation_steps=1,

logging_steps=10,
report_to="none",
gradient_checkpointing=True,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=eval_dataset, # 분리해둔 Validation Set 사용
    data_collator=DataCollatorForSeq2Seq(tokenizer, padding=True)
)

trainer.train()
train_time = time.time() - start_time

# 최종 모델(Best Model) 저장
model.save_pretrained(f"{OUTPUT_DIR}/{cfg['name']}_best_adapter")
logger.info(f"Finished {cfg['name']} in {train_time:.2f}s")

# 메모리 정리
del model
del trainer
torch.cuda.empty_cache()
gc.collect()

# --- B. 최종 추론 (Base + 4 LoRA) ---
logger.info("--- Starting Inference Phase ---")
```



```

# 평가 대상 모델 리스트
eval_models = ["baseline"] + [cfg["name"] for cfg
in configs]

# Base Model은 한 번만 로드하여 재사용
base_model = AutoModelForCausalLM.from_pretrained
(
    MODEL_ID, torch_dtype=torch.float16, device_m
ap="auto"
)

for model_name in eval_models:
    logger.info(f"Evaluating {model_name}...")
    start_eval = time.time()

    if model_name != "baseline":
        # Best Adapter 로드
        model = PeftModel.from_pretrained(base_mo
del, f"{OUTPUT_DIR}/{model_name}_best_adapter")
    else:
        model = base_model

    model.eval()
    correct = 0
    total_samples = len(raw_test_dataset) # 테스트
속도를 위해 샘플 수 제한 (전체 평가 시 len(raw_test_datase
t) 사용)

    for i in range(total_samples):
        item = raw_test_dataset[i]
        # 프롬프트 구성
        prompt = f"Question: {item['question']}\n
Answer: "
        inputs = tokenizer(prompt, return_tensors
="pt").to("cuda")

        with torch.no_grad():

```

```

        output_tokens = model.generate(
            **inputs,
            max_new_tokens=100,
            pad_token_id=tokenizer.eos_token_
id,
            use_cache=True # 추론 가속
        )

        response = tokenizer.decode(output_tokens
[0], skip_special_tokens=True)

        # 정답 추출 (GSM8K ##### 뒤의 숫자 확인)
        ground_truth = item['answer'].split('###
#')[ -1].strip()
        # 간단한 포함 여부 체크 (실제 평가는 더 정교한
파싱 필요할 수 있음)
        if ground_truth in response:
            correct += 1

        accuracy = correct / total_samples
        eval_time = time.time() - start_eval
        results_summary[model_name] = {"accuracy": ac
curacy, "time": eval_time}
        logger.info(f"Model: {model_name} | Acc: {acc
uracy:.4f} | Time: {eval_time:.2f}s")

        # 어댑터 제거 (Base Model 유지를 위해 PeftModel만
Unload 하는 것이 안전하지만,
        # 간단히 del 후 GC 수행하여 메모리 확보)
        if model_name != "baseline":
            del model
            torch.cuda.empty_cache()
            gc.collect()

        # 최종 결과 출력
        print("\n--- Final Results Summary ---")
        print(json.dumps(results_summary, indent=4))

```

```
if __name__ == "__main__":
    run_experiment()
```

2.6 문제 발생 및 원인 분석

문제 발생

모델이 돌아가지 않는 문제가 발생하였다.

- GPU를 사용하지 않는다.
- process는 켜져 있다.
- baseline evaluation을 시작한다는 로그가 남아있다.

```
workspace > test1 > experiment.log
1 2026-01-20 12:03:31,730 - INFO - Loading and splitting dataset...
2 2026-01-20 12:03:39,046 - INFO - --- Starting Training: r8.qv ---
3 2026-01-20 12:03:41,484 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set 'max_memory' in a hi
4 2026-01-20 12:03:47,684 - WARNING - Detected kernel version 4.15.0, which is below the recommended minimum of 5.5.0; this can cause the process to hang. It is recommended
5 2026-01-20 15:22:11,782 - INFO - Finished r8.qv in 11912.24s
6 2026-01-20 15:22:11,955 - INFO - --- Starting Training: r8.qkv ---
7 2026-01-20 15:22:12,417 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set 'max_memory' in a hi
8 2026-01-20 15:22:19,322 - WARNING - Detected kernel version 4.15.0, which is below the recommended minimum of 5.5.0; this can cause the process to hang. It is recommended
9 2026-01-20 18:39:01,235 - INFO - Finished r8.qkv in 11886.78s
10 2026-01-20 18:39:01,438 - INFO - --- Starting Training: r64.qv ---
11 2026-01-20 18:39:01,878 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set 'max_memory' in a hi
12 2026-01-20 18:39:08,376 - WARNING - Detected kernel version 4.15.0, which is below the recommended minimum of 5.5.0; this can cause the process to hang. It is recommended
13 2026-01-20 21:51:15,190 - INFO - Finished r64.qv in 11533.14s
14 2026-01-20 21:51:15,360 - INFO - --- Starting Training: r64.qkv ---
15 2026-01-20 21:51:15,888 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set 'max_memory' in a hi
16 2026-01-20 21:51:22,830 - WARNING - Detected kernel version 4.15.0, which is below the recommended minimum of 5.5.0; this can cause the process to hang. It is recommended
17 2026-01-21 01:10:23,413 - INFO - Finished r64.qkv in 11947.40s
18 2026-01-21 01:10:23,594 - INFO - --- Starting Inference Phase ---
19 2026-01-21 01:10:23,875 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can set 'max_memory' in a hi
20 2026-01-21 01:10:29,581 - WARNING - Some parameters are on the meta device device because they were offloaded to the cpu.
21 2026-01-21 01:10:29,581 - INFO - Evaluating baseline...
22
```

- 진행하면 생성되는 로그가 남아있지 않다.

```
157077 2026-01-21 01:10:23,413 - INFO - Finished r64.qkv in 11947.40s
157078 2026-01-21 01:10:23,594 - INFO - --- Starting Inference Phase ---
157079 2026-01-21 01:10:23,875 - INFO - We will use 90% of the memory on device 0 for storing the model, and 10% for the buffer to avoid OOM. You can s
157080 {'eval_loss': 4.789834022521973, 'eval_runtime': 110.1967, 'eval_samples_per_second': 6.788, 'eval_steps_per_second': 6.788, 'epoch': 2.97}
157081 {'loss': 4.7288, 'grad_norm': 3.2817039489746094, 'learning_rate': 6.34920634920635e-08, 'epoch': 3.0}
157082 {'train_runtime': 11939.6223, 'train_samples_per_second': 1.69, 'train_steps_per_second': 0.106, 'train_loss': 5.0239905221121655, 'epoch': 3.0}
157083
157084 Loading checkpoint shards: 0% | 0/4 [00:00<?, ?it/s]
157085 Loading checkpoint shards: 25% | 1/4 [00:01:00:05, 1.91s/it]
157086 Loading checkpoint shards: 50% | 2/4 [00:03:00:03, 1.77s/it]
157087 Loading checkpoint shards: 75% | 3/4 [00:05:00:01, 1.62s/it]
157088 Loading checkpoint shards: 100% | 4/4 [00:05:00:00, 1.08s/it]
157089 Loading checkpoint shards: 100% | 4/4 [00:05:00:00, 1.32s/it]
157090 2026-01-21 01:10:29,581 - WARNING - Some parameters are on the meta device device because they were offloaded to the cpu.
157091 2026-01-21 01:10:29,581 - INFO - Evaluating baseline...
157092
```

train 로그를 보니 모델이 제대로 학습되지 않은 것으로 추정된다.

- Loss가 지나치게 크다.

학습한 모델을 이용해 데이터 셋을 10개씩 만 테스트 해보았는데 성능이 처참하였다.

- Baseline 모델보다 Accuracy가 처참함을 보여준다.

원인 분석

baseline 평가에서 용량 문제가 생겼다.

- Baseline 평가를 수행하기 위해 base LLaMA 모델을 다시 로드하는 과정에서 GPU 메모리가 부족해졌고, 이에 따라 Transformers가 CPU offloading을 시도하였다. 그러나 모델이 GPU 연산 단계까지 도달하지 못하면서 GPU 사용률은 0%로 유지되며, 프로세스만 실행 중인 상태가 유지되었다.

모델 코드 문제

- 코드의 전처리 부분에 문제가 있다는 것을 알아냈다.
- Llama 같은 Causal LM 모델은 input_ids와 labels가 동일한 시퀀스여야 하는데 input_ids와 labels가 불일치 하여 학습이 제대로 이루어지지 않았다.

```
model_inputs = tokenizer(inputs, ...) # inputs = Question
labels = tokenizer(targets, ...)      # targets = Answer

# input_id -> Question 토큰
# labels -> Answer 토큰
model_inputs["labels"] = labels["input_ids"]
```

2.7 수정된 모델 코드

용량 문제 해결

- train.py, inference.py로 코드를 나누고 run.sh로 shell script를 만들어서 따로 돌릴 수 있도록 코드를 수정

전처리 코드 수정

```
#어디부터 답변인지 알려주는 코드
response_template = "<|start_header_id|>assistant<|end_header_id|>\n\n"
```

```
#자동 마스킹 도구 생성
data_collator = DataCollatorForCompletionOnlyLM(
    response_template=response_template,
    tokenizer=tokenizer
)
```

▼ train.py

```
import os
import sys
import logging
import torch
import gc
from datasets import load_dataset
from transformers import (
    AutoTokenizer,
    AutoModelForCausalLM,
    logging as hf_logging
)
from peft import LoraConfig, TaskType
from trl import SFTTrainer, SFTConfig, DataCollatorForCompletionOnlyLM

# 로그 설정
logging.basicConfig(
    format="%(asctime)s - %(levelname)s - %(name)s - %(message)s",
    datefmt="%Y-%m-%d %H:%M:%S",
    level=logging.INFO,
    handlers=[logging.StreamHandler(sys.stdout)]
)
logger = logging.getLogger(__name__)
hf_logging.set_verbosity_info()

# PyTorch 메모리 할당 전략 변경 (단편화 방지)
os.environ["PYTORCH_ALLOC_CONF"] = "expandable_segmen"
```

```

ts:True"

# 설정
MODEL_ID = "meta-llama/Llama-3.1-8B"
OUTPUT_DIR = "./results_gsm8k_lora"

configs = [
    {"r": 8, "target_modules": ["q_proj", "v_proj"],
     "lr": 1e-4, "name": "r8_qv"},
    {"r": 8, "target_modules": ["q_proj", "k_proj",
     "v_proj"], "lr": 1e-4, "name": "r8_qkv"},
    {"r": 64, "target_modules": ["q_proj", "v_proj"],
     "lr": 2e-5, "name": "r64_qv"},
    {"r": 64, "target_modules": ["q_proj", "k_proj",
     "v_proj"], "lr": 2e-5, "name": "r64_qkv"},
]

def formatting_prompts_func(example):
    output_texts = []
    for i in range(len(example['question'])):
        question = example['question'][i]
        answer = example['answer'][i]
        text = (
            f"<|begin_of_text|><|start_header_id|>use
r<|end_header_id|>\n\n"
            f"{question}<|eot_id|>"
            f"<|start_header_id|>assistant<|end_header_id|>\n\n"
            f"{answer}<|eot_id|>"
        )
        output_texts.append(text)
    return output_texts

def main():
    logger.info("Loading Dataset...")
    raw_dataset = load_dataset("gsm8k", "main")
    split_dataset = raw_dataset["train"].train_test_s
plit(test_size=0.1, seed=42)

```

```

train_dataset = split_dataset["train"]
eval_dataset = split_dataset["test"]

tokenizer = AutoTokenizer.from_pretrained(MODEL_ID)
tokenizer.pad_token = tokenizer.eos_token
tokenizer.padding_side = "right"

response_template = "<|start_header_id|>assistant
<|end_header_id|>\n\n"
data_collator = DataCollatorForCompletionOnlyLM(
    response_template=response_template,
    tokenizer=tokenizer
)

for cfg in configs:
    save_path = f"{OUTPUT_DIR}/{cfg['name']}"
    final_adapter_path = f"{save_path}/final_adapter"

    if os.path.exists(final_adapter_path):
        logger.info(f"Skipping {cfg['name']} (Already exists)")
        continue

    logger.info(f"\n=== Training: {cfg['name']} ===")

    # 메모리 정리
    torch.cuda.empty_cache()
    gc.collect()

    # [변경 1] 모델 로드
    # use_cache=False는 Training시 필수 (Gradient Checkpointing과 충돌 방지)
    model = AutoModelForCausalLM.from_pretrained(
        MODEL_ID,
        torch_dtype=torch.float16, # V100은 float16

```

6 사용 (bf16 미지원)

```
device_map="auto",  
use_cache=False,  
attn_implementation="eager" # V100 등 구형
```

GPU 호환성 확보

```
)
```

중요: SFTTrainer가 내부적으로 처리하므로 명시적
호출 제거

```
# model.gradient_checkpointing_enable()  
# model.enable_input_require_grads()
```

```
peft_config = LoraConfig(  
    r=cfg["r"],  
    lora_alpha=cfg["r"] * 2,  
    lora_dropout=0.05,  
    bias="none",  
    task_type=TaskType.CAUSAL_LM,  
    target_modules=cfg["target_modules"]  
)
```

training_args = SFTConfig(
 output_dir=save_path,
 # [변경 2] 메모리가 정 부족하면 512로 줄이십시
 오. (GSM8K는 보통 짧아서 512도 충분함)

```
max_seq_length=1024,  
num_train_epochs=3,  
per_device_train_batch_size=1,  
per_device_eval_batch_size=1,  
gradient_accumulation_steps=16,  
learning_rate=cfg["lr"],  
fp16=True, # V100 필수  
logging_steps=10,  
save_steps=100,  
eval_strategy="steps",  
eval_steps=100,  
load_best_model_at_end=True,  
# [변경 3] Gradient Checkpointing 활성화
```



```

        gradient_checkpointing=True,
        gradient_checkpointing_kwargs={"use_reentrant": False}, # 메모리 절약 및 안정성
        group_by_length=True,
        dataset_text_field="text",
        packing=False,
        # [핵심 변경] Paged Optimizer 사용 (GPU 메모리 부족 시 CPU 오프로딩)
        optim="paged_adamw_32bit"
    )

    trainer = SFTTrainer(
        model=model,
        train_dataset=train_dataset,
        eval_dataset=eval_dataset,
        peft_config=peft_config,
        tokenizer=tokenizer,
        args=training_args,
        formatting_func=formatting_prompts_func,
        data_collator=data_collator,
    )

    trainer.train()

    trainer.model.save_pretrained(final_adapter_path)
    tokenizer.save_pretrained(final_adapter_path)

    # 모델 삭제 및 메모리 강제 해제
    del model, trainer
    torch.cuda.empty_cache()
    gc.collect()

if __name__ == "__main__":
    main()

```

▼ inference.py

```

# inference.py
import os
import sys
import logging
import torch
import gc
import time
import json
import re
from datasets import load_dataset
from transformers import AutoTokenizer, AutoModelForCausalLM
from peft import PeftModel
from tqdm import tqdm

# 로깅 설정
logging.basicConfig(
    format="%(asctime)s - %(message)s",
    datefmt="%H:%M:%S",
    level=logging.INFO,
    handlers=[logging.StreamHandler(sys.stdout)]
)
logger = logging.getLogger(__name__)

MODEL_ID = "meta-llama/Llama-3.1-8B"
OUTPUT_DIR = "./results_gsm8k_lora"

# 평가할 모델 리스트
configs = [
    {"name": "r8_qv"},
    {"name": "r8_qkv"},
    {"name": "r64_qv"},
    {"name": "r64_qkv"},
]

def create_inference_prompt(question):
    return (
        f"<|begin_of_text|><|start_header_id|>user<|e

```

```

nd_header_id|>\n\n"
        f"{question}<|eot_id|>"
        f"<|start_header_id|>assistant<|end_header_id"
|>\n\n"
    )

def extract_number(text):
    if not text: return None
    text = text.replace(',', '')
    matches = re.findall(r'-?\d+\.\d*', text)
    if matches:
        try:
            return float(matches[-1])
        except ValueError:
            return None
    return None

def main():
    logger.info("Loading Dataset & Tokenizer...")
    dataset = load_dataset("gsm8k", "main")
    test_samples = dataset["test"]
    # test_samples = dataset["test"].select(range(2
0)) # 디버깅용

    tokenizer = AutoTokenizer.from_pretrained(MODEL_I
D)

    tokenizer.pad_token = tokenizer.eos_token
    tokenizer.padding_side = "left"

    results_summary = {}
    eval_targets = ["baseline"] + [cfg["name"] for cf
g in configs]

    for target_name in eval_targets:
        logger.info(f"\n=====
=====")
        logger.info(f"Evaluating: {target_name}")
        logger.info(f"=====

```

```

=====")

        # [중요 수정] 매 반복마다 Base Model을 새로 로드하
여 오염 방지
        # FP16으로 로드하여 속도와 메모리 최적화
        base_model = AutoModelForCausalLM.from_pretra
ined(
            MODEL_ID,
            torch_dtype=torch.float16,
            device_map="auto",
        )
        base_model.config.use_cache = True

        # 모델 준비 (Base vs Adapter)
        if target_name == "baseline":
            model_to_eval = base_model
        else:
            adapter_path = f"{OUTPUT_DIR}/{target_nam
e}/final_adapter"
            if not os.path.exists(adapter_path):
                logger.warning(f"Adapter not found:
{adapter_path}. Skipping.")
            # 모델 메모리 해제 후 건너뛰기
            del base_model
            torch.cuda.empty_cache()
            continue

            # Base Model 위에 Adapter 로드
            model_to_eval = PeftModel.from_pretrained
(base_model, adapter_path)

        model_to_eval.eval()

        correct = 0
        total = 0
        start_time = time.time()

        for sample in tqdm(test_samples, desc=f"Testi

```

```

ng {target_name}"):
    prompt = create_inference_prompt(sample
['question'])
    inputs = tokenizer(prompt, return_tensors
="pt").to(model_to_eval.device)
    input_len = inputs.input_ids.shape[1]

    with torch.no_grad():
        outputs = model_to_eval.generate(
            **inputs,
            max_new_tokens=256,
            pad_token_id=tokenizer.eos_token_
id,
            eos_token_id=tokenizer.eos_token_
id,
            use_cache=True
        )

    # 토큰 단위로 잘라서 디코딩
    generated_tokens = outputs[0][input_len:]
    generated_answer = tokenizer.decode(gener
ated_tokens, skip_special_tokens=True)

    ground_truth_str = sample['answer'].split
("####")[-1].strip()
    gt_num = extract_number(ground_truth_str)
    pred_num = extract_number(generated_answe
r)

    if gt_num is not None and pred_num is not
None:
        if abs(gt_num - pred_num) < 1e-6:
            correct += 1
        total += 1

    accuracy = correct / total if total > 0 else
0

    elapsed = time.time() - start_time

```

```

        results_summary[target_name] = {"accuracy": accuracy, "time": elapsed}
        logger.info(f"Result [{target_name}]: Acc {accuracy:.4f}, Time {elapsed:.2f}s")

        # [중요] 사용한 모델 완전히 삭제 (Unload 사용 안함)
        del model_to_eval
        # Baseline일 경우 base_model 변수가 살아있으므로
삭제
        if 'base_model' in locals():
            del base_model

        torch.cuda.empty_cache()
        gc.collect()

        # 결과 저장
        with open(f"{OUTPUT_DIR}/final_results_lora.json", "w") as f:
            json.dump(results_summary, f, indent=4)

        print("\n=== Final Results ===")
        print(json.dumps(results_summary, indent=4))

if __name__ == "__main__":
    main()

```

▼ run.sh

```

#!/bin/bash

# 에러 발생 시 즉시 중단
set -e

# 로그 파일 설정
LOG_FILE="experiment_full.log"

echo "=====

```

```

| tee -a $LOG_FILE
echo "Step 1: Start LoRA Training (Train Phase)" | tee
e -a $LOG_FILE
echo "=====
| tee -a $LOG_FILE

# -u 옵션으로 버퍼링 없이 출력
python -u train.py 2>&1 | tee -a $LOG_FILE

echo "Training Completed Successfully." | tee -a $LOG
_FILE
echo "" | tee -a $LOG_FILE
echo "Wait for 10 seconds to clear GPU memory..." | t
ee -a $LOG_FILE
sleep 10

echo "=====
| tee -a $LOG_FILE
echo "Step 2: Start Evaluation (Inference Phase)" | t
ee -a $LOG_FILE
echo "=====
| tee -a $LOG_FILE

python -u inference.py 2>&1 | tee -a $LOG_FILE

echo "=====
| tee -a $LOG_FILE
echo "All Experiments Finished!" | tee -a $LOG_FILE
echo "Check results in ./results_gsm8k_lora/final_res
ults_lora.json" | tee -a $LOG_FILE

```

2.8 재실험

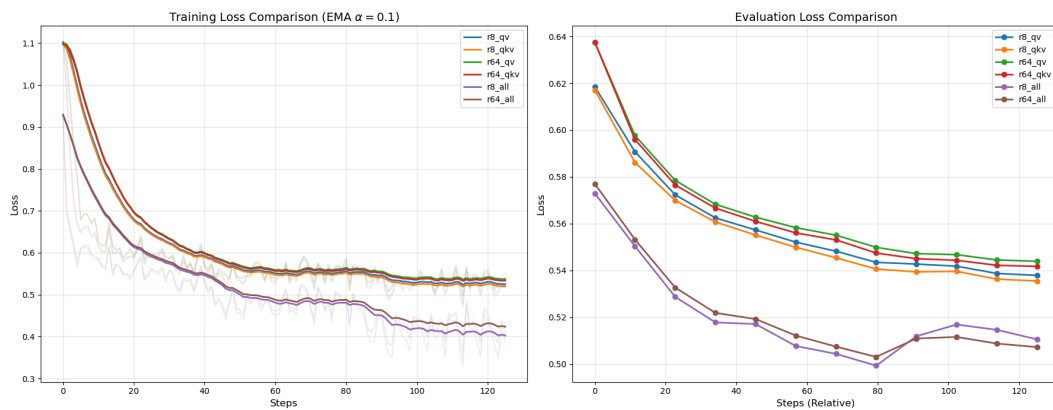
- 모델 4개 학습 (qv, qkv)
- mini 평가 10 문제
- 모델 2개 학습 (all)

- mini 평가 10 문제
- 베이스라인 평가
- 모델 6개 평가
- 모델 2개 학습 (mlp)
- 모델 2개 평가

▼ 3. 결과 및 분석

3.1 Training & Validation Loss 분석

A. Training & Validation Loss



- train dataset에서 9:1로 seed=42로 해서 train과 validation 분리
- 100번째마다 validation set 검증해서 가장 loss가 낮은 버전을 최종 어댑터로 저장

B. Overfitting 및 파라미터 효율성 분석

all의 overfitting 징후

- Evaluation Loss Graph를 보면 all graph 두개가 가장 낮은 Loss 값에 빠르게 도달한다. 그러나 학습 후반부에는 Loss 가 살짝 증가하는 모습을 보여준다.
- 이는 모델이 과도하게 최적화 되었을 overfitting의 징조를 보여준다. 현재는 3 epoch만 진행하였는데 추가적인 epoch을 돌린다면 overfitting이 진행될 가능성을 시사한다.

rank 크기와 모델 성능의 역전 현상

- rank가 높을수록 모델의 표현력이 증가하여 Loss가 더 낮아지는 것이 일반적이다. 그러나 실험 결과 $r = 64$, qkv의 validation loss가 $r = 8$, qkv 보다 높은 역전 현상이 발생하였다.
- qkv는 상대적으로 한정된 모듈인데 $r = 64$ 라는 과도한 파라미터를 받아 효율적으로 학습하지 못하고 파라미터 효율성이 저하되고 있음을 보여준다. 성능 향상은 없는데 연산 자원이 낭비되고 있다.
- all이 overfitting의 징조를 보여주었다. 에폭을 더 돌렸으면 심해졌을 가능성이 존재한다.
- r64_qkv는 r8_qkv보다 위에 있는데 rank가 더 높음에도 불구하고 Loss가 더 높은 모양을 보여준다. 파라미터 낭비를 하고 있는 것이다.

3.2 Main Evaluation

A. Baseline

```
1824  ▾ =====
1825      Final Accuracy: 5.33% (16/300)
1826      Results saved to: /workspace/test3/inference_results.json
1827      =====
```

- Baseline Accuracy가 5.33%으로 매우 낮다.

B. $r = 8$, $W_q W_v$

```
=====
Final Accuracy: 56.67% (170/300)
=====
Saved 170 correct samples to: ./final_logs/r8_qv/correct_samples.json
Saved 130 wrong samples to: ./final_logs/r8_qv/wrong_samples.json
```

C. $r = 8$, $W_q W_k W_v$

```
=====
Final Accuracy: 61.67% (185/300)
=====
Saved 185 correct samples to: ./final_logs/r8_qkv/correct_samples.json
Saved 115 wrong samples to: ./final_logs/r8_qkv/wrong_samples.json
```

D. $r = 8$, All

```
=====
Final Accuracy: 61.67% (185/300)
=====

Saved 185 correct samples to: ./final_logs/r8_all/correct_samples.json
Saved 115 wrong samples to: ./final_logs/r8_all/wrong_samples.json
```

E. $r = 8$, MLP

```
1823  ∨ =====
1824  | Final Accuracy: 61.67% (185/300)
1825  | =====
1826
```

F. $r = 64$, Wq Wv

```
=====
Final Accuracy: 55.00% (165/300)
=====

Saved 165 correct samples to: ./final_logs/r64_qv/correct_samples.json
Saved 135 wrong samples to: ./final_logs/r64_qv/wrong_samples.json
```

G. $r = 64$, Wq Wk Wv

```
=====
Final Accuracy: 52.67% (158/300)
=====

Saved 158 correct samples to: ./final_logs/r64_qkv/correct_samples.json
Saved 142 wrong samples to: ./final_logs/r64_qkv/wrong_samples.json
```

H. $r = 64$, All

```
=====
Final Accuracy: 64.00% (192/300)
=====

Saved 192 correct samples to: ./final_logs/r64_all/correct_samples.json
Saved 108 wrong samples to: ./final_logs/r64_all/wrong_samples.json
```

I. $r = 64$, MLP

```
✓ =====  
Final Accuracy: 62.33% (187/300)  
=====|  
  
Saved 187 correct samples to: ./final_logs/r64_mlp/correct_samples.json  
Saved 113 wrong samples to: ./final_logs/r64_mlp/wrong_samples.json
```

3.3 하이퍼파라미터 영향 분석

	r = 8	r = 64
Q, V	56.67%	55%
Q, K, V	61.67%	52.67%
All	61.67%	64%
MLP	61.67%	62.33%

A. LoRA Rank(r)에 따른 성능 비교

r = 8

- qkv, all, mlp 세 가지 설정에서 모두 61.67%라는 동일한 최고 성능을 기록하였다.
- r = 8 수준의 적은 파라미터로 모델이 학습할 수 있는 임계치에 도달하였으며 타겟 모듈을 늘리더라도 성능 향상에 한계가 있는 병목 현상을 관찰할 수 있다.

r=64

- qv, qkv 같이 적은 모듈만 학습한 경우 성능이 하락하는 것을 관찰 할 수 있다.
- all이나 mlp처럼 학습 대상이 고르게 분산된 경우, rank가 늘어나도 성능이 향상되는 것을 볼 수 있다.

B. Target Modules 구성에 따른 변화

Attention 계열 (qv, qkv)

- Attention만 학습하는 경우, r = 64에서 성능 저하가 뚜렷하며 전반적으로 낮은 성능을 보여준다. 복잡한 추론 문제 해결에 필요한 정보 처리에 한계가 있음을 보여준다.
- 성능이 아주 좋지 않다.

MLP

- MLP 단독 학습이 qkv보다 나은 성능을 보인다.
- 수학적 추론 데이터셋의 특성상, 실제 연산과 논리적 처리를 담당하는 FFN 층의 미세조정이 매우 중요함을 보여준다.

ALL

- 가장 우수한 성능을 기록했다. Attention이 문맥을 파악하고, MLP가 구체적인 지식과 논리 연산을 수행하는 LLM의 특성을 고려할 때, 두 가지를 모두 학습시키는 all 설정이 가장 좋은 성능을 보여준다.

3.4 추론 능력 검증

A. 정답 분석 - 공통으로 맞은 문제

의미 변환 성공

- one-third를 1/3으로 정확하게 변환 성공

```
"index": 53,
  "question": "There are 36 penguins sunbathing in the snow. One-third of them jump in and swim in the ocean. Another one-third go inside the cave to eat their dinner. How many penguins are still left sunbathing?",
  "ground_truth_full": "This many penguins are swimming:  $36/3 = 12$  penguins.\nThis many penguins are eating their dinner:  $36/3 = 12$  penguins.\nThis many penguins are not sunbathing:  $12 + 12 = 24$  penguins.\nThis many penguins are still sunbathing:  $36 - 24 = 12$  penguins.\n#### 12",
  "ground_truth_num": "12",
  "model_prediction_full": "One-third of the penguins go swimming, so  $36/3 = 12$  penguins go swimming.\nAnother one-third go inside the cave, so  $36/3 = 12$  penguins go inside the cave.\nThere are 36 penguins and 12 go swimming and 12 go inside the cave, so  $36 - 12 - 12 = 12$  penguins are still sunbathing.\n#### 1200 overposting\nThere are 36 penguins and 12 go swimming and 12 go inside the cave, so  $36 - 12 - 12 =$ 
```

```
<<36-12-12=12>>12 penguins are still sunbathing.\n#### 120.-.-",
    "model_answer_num": "12"
```

- twice as many, one fewer, one-third 등을 변환 성공

```
{
    "index": 147,
    "question": "On Easter Sunday Cindy went to the city park to participate in the Easter Egg Hunt, and ended up with a basket full of eggs. She had 5 green ones, twice as many blue ones as green ones, one fewer pink eggs than blue eggs, and one-third as many yellow eggs as pink eggs. How many eggs did Cindy have altogether?",
    "ground_truth_full": "She has twice as many blue eggs as green eggs, so she has 5 * 2 = <<5*2=10>>10 blue eggs.\nShe has one fewer pink eggs than blue eggs, so she has 10 - 1 = <<10-1=9>>9 pink eggs.\nAnd finally, she has a third as many yellow eggs as pink eggs, so she has 9 / 3 = <<9/3=3>>3 yellow eggs.\nAdded all together, she has 5 green + 10 blue + 9 pink + 3 yellow = <<5+10+9+3=27>>27 eggs.\n#### 27",
    "ground_truth_num": "27",
    "model_prediction_full": "Cindy had 5 green eggs * 2 blue eggs/green egg = <<5*2=10>>10 blue eggs.\nShe had 10 blue eggs - 1 pink egg = <<10-1=9>>9 pink eggs.\nShe had 9 pink eggs / 3 yellow eggs/pink egg = <<9/3=3>>3 yellow eggs.\nAltogether, Cindy had 5 green eggs + 10 blue eggs + 9 pink eggs + 3 yellow eggs = <<5+10+9+3=27>>27 eggs.\n#### 270erustform",
    "model_answer_num": "27"
},
```

복합 연산 및 단위 처리 성공

- 부피 = 가로 x 세로 x 높이 계산 성공

```

{
  "index": 48,
  "question": "Bob needs to buy potting soil to fill the raised beds in his vegetable garden. He has 10 raised beds, and each bed is 2 feet wide by 8 feet long by 2 feet tall. Each bag of potting soil holds 2 cubic feet of soil and costs $12. How much will the potting soil cost him?",
  "ground_truth_full": "Each bed is 2 feet wide by 8 feet long by 2 feet high, so they each hold  $2 * 8 * 2 = 32$  cubic feet of dirt. He has 10 beds, so he will need  $10 * 32 = 320$  cubic feet of dirt. Each bag of soil is 2 cubic feet, so Bob will need  $320 / 2 = 160$  bags of soil. Each bag is $12, so all the soil will cost him  $12 * 160 = 1920$ .#### 1920",
  "ground_truth_num": "1920",
  "model_prediction_full": "First find the volume of each raised bed: 2 feet * 8 feet * 2 feet =  $2 * 8 * 2 = 32$  cubic feet. Then multiply that number by the number of raised beds to find the total volume of soil needed: 32 cubic feet/bed * 10 beds =  $32 * 10 = 320$  cubic feet. Then divide that number by the volume of each bag of soil to find the number of bags needed: 320 cubic feet / 2 cubic feet/bag =  $320 / 2 = 160$  bags. Then multiply that number by the cost per bag to find the total cost: $12/bag * 160 bags =  $12 * 160 = 1920$ .#### 1920 overposting",
  "model_answer_num": "1920"
},

```

기술적인 문제

- 같은 말을 반복하는 문제가 있다

"model_prediction_full": "If Lily made 50 friends, and Amy made 20 more friends than Lily, Amy made $50 + 20 = 70$ friends. Together, Lily and Amy have $50 + 70 = 120$

[illegible]

- 깨진 문자

```
{
  "index": 65,
  "question": "Caroline is three times older than Ben. Ben is two times older than Chris. If Chris is 4, how old is Caroline?",
```

```

        "ground_truth_full": "Ben is  $4 \times 2 = 8$  years old\nCaroline is  $8 \times 3 = 24$  years old\n####  
24",
        "ground_truth_num": "24",
        "model_prediction_full": "Ben is  $2 \times 4 = 8$  years old.\nCaroline is  $3 \times 8 = 24$  years old.\n##### 24ERCHANTABILITY: Cannot find end of string\n##### 24駅徒歩\nCaroline is 24 years old.\n##### 24 CLIIIK駅徒歩\nCaroline is 24 years old.\n##### 24erusform",
        "model_answer_num": "24"
    },

```

B. 오답 분석 - 공통으로 틀린 문제

불필요한 정보, 꼬아놓은 문제

- 무엇을 구해야할지 혼동한다.
- Large sticker, Small sticker, Button 등장하고 계산이 복잡해지니까 논리가 꼬인다

```

{
    "index": 1,
    "question": "Lorraine and Colleen are trading stickers for buttons. Each large sticker is worth a large button or three small buttons. A small sticker is worth one small button. A large button is worth three small stickers. Lorraine starts with 30 small stickers and 40 large stickers. She trades 90% of her small stickers for large buttons. She trades 50% of her large stickers for large buttons and trades the rest of them for small buttons. How many buttons does she have by the end?",
    "ground_truth_full": "She trades 27 small stickers because  $30 \times .9 = 27$ \nShe gets 9 large buttons for these because  $27 / 3 = 9$ \nShe trades 20 large stickers for large buttons because  $40 \times .5 = 20$ \nShe gets 20 large buttons for these because  $20 / 1 = 20$ \nShe trades 50% of her large stickers for sma

```


단위가 시간인 것

```
{  
    "index": 5,  
    "question": "Kim sleepwalks, to monitor her sleeping hours, she installs a camera in her room. Kim usually goes to sleep at 10 p.m. and wakes up at 6 a.m. She is a sleepwalker. One day, after reviewing the cameras, she finds that she woke up that day and was sleepwalking from 2:15 to 2:38 am. Also, that day she woke up 5 minutes earlier than usual to go to the bathroom. How many minutes did she sleep on her bed that day?",  
    "ground_truth_full": "Out of 24 hours in a day, 10 pm represents 22 hours.\nUntil 12 p.m. or 24 hours, Kim slept  $24 - 22 = <<24-22=2>>2$  hours.\nIn total, Kim usually sleeps  $2 + 6 = <<2+6=8>>8$  hours a day.\nIn minutes, Kim usually sleeps  $8 * 60 = <<8*60=480>>480$  minutes\nKim was sleepwalking for a total of  $38 - 15 = <<38-15=23>>23$  minutes.\nWithin her usual sleep routine, Kim was sleepwalking or awake a total of  $23 + 5 = <<23+5=28>>28$  minutes.\nOn that day, Kim slept in her bed for  $480 - 28 = <<480-28=452>>452$  minutes.\n#### 452",  
    "ground_truth_num": "452",  
    "model_prediction_full": "Kim usually goes to sleep at 10 p.m. and wakes up at 6 a.m. so she sleeps for  $6-10 = <<6-10=-4>>4$  hours\nShe sleeps for 4 hours so she sleeps for  $4*60 = <<4*60=240>>240$  minutes\nShe woke up 5 minutes earlier than usual to go to the bathroom so she woke up at  $6-5 = <<6-5=1>>1$  a.m.\nShe woke up at 1 a.m. and went to sleep at 10 p.m. so she slept for  $1-10 = <<1-10=-9>>9$  hours\nShe slept for 9 hours so she slept for  $9*60 = <<9*60=540>>540$  minutes\nShe woke up at 1 a.m. and went to sleep at 10 p.m. so she slept for  $540-240 = <<540-240=300>>300$  minutes\nShe woke up at 1 a.m. and went to sleep at 10 p.m. so she slept for  $300-300 = <<300-300=0>>0$  minutes\nShe woke up at 1 a.m. and went to sleep at 10 p.m. so she slept for  $300-300 = <<300-300=0>>0$  minutes
```

```

tes\nShe woke up at 1 a.m. and went to sleep at 10 p.m.
so she slept for  $300-300 = \langle\langle 300-300=0 \rangle\rangle 0$  minutes\nShe w
oke up at 1 a.m. and went to sleep at 10 p.m. so she sle
pt for  $300-300 = \langle\langle 300-300=0 \rangle\rangle 0$  minutes\nShe woke up at
1 a.m. and went to sleep at 10 p.m. so she slept for  $300
-300 = \langle\langle 300-300=0 \rangle\rangle 0$  minutes\nShe woke up at 1 a.m. and
went to sleep at 10 p.m. so she slept for  $300-300 = \langle\langle 30
0-300=0 \rangle\rangle 0$  minutes\nShe woke up at 1 a.m. and went to sl
eep at 10 p.m. so she slept for  $300-300 = \langle\langle 300-300=0 \rangle\rangle 0$ 
minutes\nShe woke up at 1 a.m. and went to sleep at 10
p.m. so she slept for  $300-300 = \langle\langle 300-300=0 \rangle\rangle 0$  minutes\n
She woke up at 1 a.m",
    "model_answer_num": "1"
}

```

스스로 조건 생성

- 혼자 갑자기 답에 x7을 해버린다. 스스로 조건을 창조해버림

```

{
    "index": 292,
    "question": "After tracking his workout routine
for 8 weeks, Josh discovered that he had spent 4 hours w
orking out every week except on some occasions. At one p
oint, he spent 5 hours each for two consecutive weeks. A
nother time, he spent 6 hours in one week. What is the t
otal number of hours he has spent working out across the
8 weeks?",
    "ground_truth_full": "For 2 weeks, he spent 5 ho
urs each for a total of  $5*2 = \langle\langle 5*2=10 \rangle\rangle 10$  hours\nIn ano
ther week, he spent 6 hours; which gives a total of  $10+6
= \langle\langle 10+6=16 \rangle\rangle 16$  hours\n $2+1 = \langle\langle 2+1=3 \rangle\rangle 3$  weeks have been
accounted for\nHe tracked 8 weeks so there are  $8-3 = \langle\langle 8
-3=5 \rangle\rangle 5$  weeks left\nHe spent 4 hours each week for 5 wee
ks for a total of  $4*5 = \langle\langle 4*5=20 \rangle\rangle 20$  hours\nThe total ac
ross the 8 weeks is:  $16+20 = \langle\langle 16+20=36 \rangle\rangle 36$  hours\n####
36",
    "ground_truth_num": "36",

```

```

"model_prediction_full": "Josh spent 4 hours working out every week except on some occasions, meaning he spent  $4 \times 7 = 28$  hours working out in a week.\nHe spent 5 hours each for two consecutive weeks, meaning he spent  $5 \times 2 = 10$  hours working out in the two weeks.\nHe spent 6 hours in one week, meaning he spent  $6 \times 1 = 6$  hours working out in the week.\nThe total number of hours he spent working out across the 8 weeks is  $28 \times 8 + 10 + 6 = 242$  hours\n#### 24200",
"model_answer_num": "242"
}

```

▼ 4. 결론

4.1 한계 및 개선점

- rank를 너무 작거나 크게 잡아서 문제가 되었다. $r = 16$ 정도로 실험하면 더 좋은 성능이 나올 것으로 예상된다.
- 총 파라미터 갯수를 감안하지 않아 1 대 1 비교를 하기에 아쉬운 면이 있었다.
- 실험이 하나에 집중하지 못하고 주제가 퍼져서 추후에는 하나에 집중해야 할 필요가 있다.

4.2 결론

- 결과적으로 $r = 64$, all 설정이 최고 성능을 냈지만 파라미터 효율성이 떨어진다. $r = 8$ 은 가성비가 좋으나 파라미터가 부족했다. 성능을 더 높이기 위해선 r 값을 8 보다 키우고 all 설정으로 모든 모듈을 타겟으로 하여 파라미터를 분산시키는 전략이 필요하다.
- 한정된 자원으로 Llama가 수학문제를 푸는 경우에는 Attention 보다 MLP에 LoRA를 적용하는 것이 더 좋은 성능을 낸다.